Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

DESARROLLO DE APLICACIONES MÓVILES

Examen Parcial — Unidad 2

Versión A · Caso: AgroSierra, labores de campo de una cooperativa agrícola

Dato	Detalle	Dato	Detalle
Asignatura	Desarrollo de Aplicaciones Móviles	Ciclo	VI · Semestre 2026-2
Unidad	2. Conectividad, CRUD REST y persistencia multiplataforma	Sesión	12 (evaluación de unidad)
Modalidad	Individual: producto propio y defensa individual	Duración	3 horas continuas
Plataforma	Android · backend propio	Fecha	27 de octubre de 2026
Nivel	Base · Grupo A	Puntaje	20 puntos
Docente	Benjamin David Reyna Barreto	Evaluación	Examen Parcial II · Unidad 2

Apellidos y nombres	Código	Grupo	Fecha	Nota
			27 de octubre de 2026	

1. Indicaciones generales

Este examen evalúa el Producto de la Unidad 2: «Aplicación KMP con CRUD REST completo con Ktor, autenticación JWT y persistencia local offline-first». A diferencia de la Unidad 1, aquí no se entrega ningún servicio: tú construyes el backend del caso con su seguridad y la aplicación Android que lo consume. El examen es individual: el producto, la solicitud de cambio y la defensa son de autoría propia. Es equivalente en exigencia y puntaje a la otra versión, y se diferencia en el caso de negocio y en el nivel de andamiaje que se te entrega.

Parte	Momento	Qué se evalúa	Puntaje
Parte I. Producto del caso	Sesiones 7 a 11; se demuestra en los primeros 45 minutos.	Que el backend y la aplicación cumplan los requerimientos funcionales y técnicos del caso.	12 pts

Parte	Momento	Qué se evalúa	Puntaje
Parte II. Solicitud de cambio	80 minutos en aula; cada estudiante recibe una solicitud distinta.	Capacidad individual de modificar el backend y la aplicación respetando la arquitectura.	4 pts
Parte III. Defensa técnica	35 minutos, unos 15 por estudiante, al cierre de la sesión.	Comprensión de la seguridad, del contrato del API y de la estrategia offline.	2 pts
Criterio transversal. Trazabilidad del desarrollo con Git	Toda la unidad; se verifica en el historial de los repositorios.	Que el desarrollo esté versionado y sea trazable: ramas de trabajo y commits distribuidos en el tiempo.	2 pts

1.1 Distribución del tiempo

El examen se desarrolla en una única jornada de tres horas continuas. Los tiempos son estrictos: al cerrarse cada bloque se evalúa lo que exista en tus repositorios en ese momento.

Bloque	Duración	Actividad
1	10 min	Indicaciones, verificación de que cada estudiante tiene sus repositorios actualizados, el backend levantado y el emulador operativo.
2	45 min	Parte I. Cada estudiante demuestra su backend respondiendo y su aplicación Android consumiéndolo, y recorre los requerimientos funcionales, los roles y los escenarios sin conexión.
3	80 min	Parte II. Cada estudiante implementa su solicitud de cambio en su propia rama, con un mínimo de tres commits.
4	35 min	Parte III. Defensa técnica individual sobre el código propio.
5	10 min	Cierre: subida de ramas, etiquetado del commit evaluado y entrega de evidencias.

1.2 Condiciones

- El backend debe levantarse desde el repositorio entregado con un solo comando y quedar accesible para la aplicación. Un servicio que solo existe en capturas no se califica.
- La aplicación debe ejecutarse en un emulador o dispositivo Android. Esta evaluación considera únicamente la plataforma Android.
- Se permite consultar la documentación oficial de Kotlin, Ktor, SQLDelight, Compose y del framework de servicios utilizado, así como los propios repositorios del equipo.
- El examen es individual en todas sus partes. No se permite copiar código de otro estudiante ni recibir ayuda externa, y dos entregas con el mismo código se califican ambas con cero.
- El uso de asistentes de inteligencia artificial está permitido solo si el estudiante explica y justifica cada línea durante la defensa. Si no puede explicar su propio código, esa parte se califica con cero.
- El desarrollo debe quedar versionado y ser trazable en Git: ramas de trabajo por funcionalidad y commits distribuidos a lo largo de la unidad. Las condiciones completas están en la sección 5.

- Durante el examen debes registrar como mínimo tres commits en tu rama, distribuidos a lo largo del bloque de trabajo. Un solo commit al final no acredita el proceso.
- Un repositorio creado el día de la evaluación, o con un historial de un solo commit, no acredita el desarrollo y se califica con cero en el criterio de Git.

Prohibición explícita

Esta evaluación no admite servicios simulados ni datos incrustados en la aplicación. Está prohibido devolver listas escritas a mano en el repositorio de la aplicación, dejar la autenticación sin verificar la firma del token, o autorizar las operaciones comparando cadenas de rol dentro de un controlador suelto.

Tampoco se acepta una aplicación cuya pantalla principal lea directamente de la red: la interfaz debe leer de la base de datos local. Ese es precisamente el punto de la unidad.

2. Presentación del caso

2.1 La organización

AgroSierra es una cooperativa agrícola que administra las chacras de noventa socios en tres valles. Cada chacra está dividida en lotes, y sobre cada lote se ejecutan labores a lo largo de la campaña: riegos, fumigaciones y cosechas.

Hoy los técnicos de campo anotan esas labores en una libreta y las transcriben al volver a la oficina, a veces con dos o tres días de retraso. Nadie sabe con certeza qué se aplicó a cada lote ni cuándo, y las fumigaciones repetidas por olvido han costado dos campañas.

2.2 El alcance del proyecto

La cooperativa aprobó construir el sistema completo: un servicio web que custodie la información de lotes y labores, y una aplicación Android que los técnicos usen en la chacra. El servicio no existe todavía: tu equipo debe construirlo.

Las chacras están en zonas sin cobertura, de modo que la aplicación tiene que funcionar sin conexión y sincronizar cuando el técnico vuelva a tener señal. Y como no todos los usuarios pueden hacer lo mismo, el servicio debe decidir quién puede qué: un técnico registra labores, pero solo el jefe de campo cierra la campaña de un lote.

El criterio central de este examen

No se evalúa únicamente que la aplicación muestre datos. Se evalúa que la seguridad viva en el servicio y no en la pantalla, y que la aplicación siga siendo útil cuando el técnico está en la chacra sin señal. Un sistema que solo funciona con cobertura y que confía en que el botón esté oculto no resuelve el problema de AgroSierra.

2.3 Nivel y andamiaje entregado

Esta versión está calibrada para un nivel base. Eso no reduce el puntaje ni la exigencia técnica, pero sí determina cuánto trabajo de análisis se te entrega resuelto.

- El modelo de dominio y el contrato del API se te entregan escritos en las secciones 4.5 y 4.3; debes implementarlos tal cual, respetando nombres de campo, tipos y códigos de estado.
- La matriz de roles y privilegios de la sección 3.3 está resuelta: tu tarea es implementarla, no diseñarla.
- El esquema de la base local se entrega en la sección 4.6 y los datos semilla, incluidos los usuarios de prueba, en el anexo.
- A cambio, se exige rigor en la ubicación del código: un privilegio verificado solo en el composable se califica como no logrado, aunque el botón efectivamente desaparezca.

3. Requerimientos funcionales

Cada requerimiento es verificable por observación directa durante la evaluación: el del servicio, con la herramienta de pruebas del API; el de la aplicación, en el emulador o el dispositivo.

3.1 Servicio AgroAPI

Código	Requerimiento	Comportamiento esperado
RF-B01	Inicio de sesión	POST que recibe usuario y contraseña, valida contra la base con la contraseña cifrada y devuelve un token firmado con vigencia de una hora, junto con el nombre del usuario y sus privilegios.
RF-B02	Padrón de lotes	Listado paginado de lotes con código, nombre, hectáreas, cultivo, valle y estado de campaña. Admite filtro por valle y por estado.
RF-B03	Labores de un lote	Listado paginado de las labores de un lote, ordenado de la más reciente a la más antigua, con tipo, fecha, responsable, cantidad aplicada y observación.
RF-B04	Registro de labor	Creación de una labor sobre un lote. Devuelve 201 con la labor creada y su identificador definitivo.
RF-B05	Corrección y eliminación	Actualización y eliminación de una labor, sujetas a las reglas RN-02 y a los privilegios de la sección 3.3. La eliminación responde 204 sin cuerpo.
RF-B06	Cierre de campaña	Operación que cierra la campaña de un lote. Solo el jefe de campo puede ejecutarla y debe cumplirse la regla RN-03.
RF-B07	Usuario actual	Consulta que devuelve los datos y privilegios del usuario del token, usada por la aplicación para decidir qué acciones ofrece.

3.2 Aplicación AgroCampo

Código	Requerimiento	Comportamiento esperado
RF-A01	Inicio de sesión	Formulario de usuario y contraseña, estado de carga, mensaje distinto para credenciales incorrectas y para falta de conexión, y paso al área protegida al autenticarse.
RF-A02	Sesión persistente	Al reabrir la aplicación, si el token guardado sigue vigente se entra directamente; si venció, se vuelve al inicio de sesión sin mensajes técnicos.

Código	Requerimiento	Comportamiento esperado
RF-A03	Padrón de lotes	Listado alimentado por la base de datos local, con filtro por valle, buscador por nombre o código y señal visible de cuándo fue la última sincronización.
RF-A04	Detalle del lote y sus labores	Datos del lote y sus labores, con la acción «Registrar labor» y la acción «Cerrar campaña» visible solo para quien tiene el privilegio.
RF-A05	Registro y corrección de labores	Formulario con validación, que funciona con y sin conexión, muestra los mensajes del servidor bajo el campo correspondiente y marca la fila como pendiente mientras no se haya enviado.
RF-A06	Sincronización	Acción manual de sincronizar y sincronización automática al recuperar la conexión, con indicador de progreso y resumen de lo enviado y lo recibido.
RF-A07	Perfil y cierre de sesión	Datos del usuario, su rol, la lista de privilegios que posee y el cierre de sesión que borra el token del almacenamiento seguro.

3.3 Roles y privilegios

El sistema define tres roles. Los permisos se conceden por privilegio: el rol es un conjunto de privilegios y la autorización se verifica siempre contra el privilegio, nunca contra el nombre del rol escrito en el código.

Rol	Quién es	Qué hace en el sistema
TECNICO	Técnico de campo	Recorre los lotes y registra las labores que ejecuta.
JEFE_CAMPO	Jefe de campo	Supervisa a los técnicos, corrige labores y cierra la campaña de un lote.
ADMINISTRADOR	Administrador	Gestiona usuarios y roles; no registra labores.

Matriz de privilegios. Esta tabla es el contrato de autorización del caso: debe quedar reflejada en la configuración de seguridad o en la base de datos, y la aplicación debe respetarla también en lo que ofrece al usuario.

Privilegio	Permite	TECNICO	JEFE_CAMPO	ADMIN
LOTE_LEER	Consultar el padrón y el detalle de lotes	Sí	Sí	Sí
LABOR_LEER	Consultar las labores de un lote	Sí	Sí	Sí
LABOR_CREAR	Registrar una labor nueva	Sí	Sí	No
LABOR_EDITAR	Corregir una labor existente	Solo las propias	Todas	No
LABOR_ELIMINAR	Eliminar una labor	No	Sí	No
CAMPANIA_CERRAR	Cerrar la campaña de un lote	No	Sí	No
USUARIO_GESTIONAR	Crear usuarios y asignar roles	No	No	Sí

Dónde se verifica el privilegio

La verificación obligatoria ocurre en el servicio: una petición sin el privilegio necesario debe responder 403 aunque llegue con un token válido. Ocultar el botón en la aplicación es una mejora de experiencia, no un control de seguridad.

Durante la defensa se te pedirá invocar la operación directamente contra el servicio, con el token de un usuario que no tiene el privilegio, y mostrar que el servicio la rechaza.

3.4 Reglas de negocio

Las cuatro reglas se implementan y se hacen cumplir en el servicio, en la capa que coordina la operación, no en el controlador ni en la aplicación. La aplicación debe mostrar el motivo que devuelve el servicio.

Código	Regla
RN-01	No se puede registrar ni corregir ninguna labor sobre un lote cuya campaña esté cerrada. El servicio responde 409 con el motivo.
RN-02	Un técnico solo puede corregir las labores que él mismo registró y únicamente dentro de las veinticuatro horas siguientes al registro. Pasado ese plazo, o si la labor es de otro técnico, el servicio responde 403.
RN-03	La campaña de un lote no puede cerrarse si el lote no tiene al menos una labor de tipo Cosecha registrada. El servicio responde 409.
RN-04	En una fumigación, la cantidad aplicada no puede superar la dosis máxima por hectárea del lote multiplicada por sus hectáreas. El servicio responde 400 con el detalle en el campo de cantidad.

3.5 Estados de interfaz y manejo de errores

Toda pantalla que carga datos debe mostrar los estados de carga, contenido y lista vacía. El manejo de errores es el eje de esta unidad: cada respuesta del servicio tiene una conducta distinta en la aplicación.

Respuesta del servicio	Cuándo se produce	Qué hace la aplicación
200 OK	Consulta o actualización correcta.	La capa de datos convierte el cuerpo al dominio.
201 Created	Creación correcta.	Se guarda la fila definitiva y se limpia la marca de pendiente.
204 No Content	Eliminación correcta, sin cuerpo.	No se intenta convertir el cuerpo: solo se comprueba el código.
400 Bad Request	Validación fallida, con validationErrors.	El mensaje se muestra bajo el campo que lo originó.
401 Unauthorized	Token ausente, alterado o vencido.	Se borra el token, se limpia la sesión y se vuelve al inicio de sesión.

Respuesta del servicio	Cuándo se produce	Qué hace la aplicación
403 Forbidden	Autenticado sin el privilegio requerido.	Se informa que la operación no está autorizada y la sesión se mantiene.
404 Not Found	El recurso no existe.	Se informa y se refresca el listado; no se ofrece reintentar.
409 Conflict	Regla de negocio del servidor incumplida.	Se muestra el motivo devuelto por el servidor.
500 Internal Server Error	Fallo del servidor.	Mensaje breve y botón de reintento.

4. Requerimientos técnicos obligatorios

Estos requerimientos corresponden a los criterios de evaluación del Producto de la Unidad 2 declarados en el sílabo y se verifican revisando el código fuente de ambos repositorios.

4.1 Servicio AgroAPI

Ámbito	Exigencia
Servicio REST	Servicio construido con el framework usado en el curso, con rutas versionadas bajo /api/v1, verbos HTTP coherentes y códigos de estado correctos: 200, 201, 204, 400, 401, 403, 404, 409 y 500.
Autenticación JWT	Punto de inicio de sesión que valida credenciales contra la base de datos con la contraseña cifrada, y devuelve un token firmado con vigencia de una hora. El token transporta el identificador del usuario y sus privilegios. No se implementa renovación.
Filtro de seguridad	Todas las rutas, salvo la de inicio de sesión, exigen el token. Un token ausente, alterado o vencido produce 401; un usuario autenticado sin el privilegio necesario produce 403.
Roles y privilegios	Los permisos se verifican por privilegio y no por nombre de usuario. La asignación de privilegios a cada rol vive en la base de datos o en la configuración de seguridad, no escrita dentro de los controladores.
Paginación	Los listados devuelven el envoltorio de paginación descrito en 4.2 y aceptan los parámetros de página y tamaño.
Validación	Los formularios rechazados responden 400 con el detalle de campo y mensaje, según el formato de error de 4.2.
Persistencia	Base de datos relacional con el esquema del caso y datos semilla cargados al iniciar, incluidos los usuarios de prueba de la sección 10.

4.2 Contrato común: paginación y errores

Todos los listados y todos los errores del servicio siguen este formato. La aplicación declara sus DTO en función de él, y es el motivo por el que un listado no se deserializa como una lista.

```
// Envoltorio de paginación de todos los listados
{
  "contenido": [ ... ],
  "pagina": 0, "tamanio": 20,
  "totalElementos": 37, "totalPaginas": 2, "ultima": false
}

// Formato de error de todo el servicio
{
  "timestamp": "2026-10-27T09:14:22", "status": 400,
  "error": "Bad Request", "message": "Datos invalidos",
  "path": "/api/v1/...",
  "validationErrors": { "campo": "mensaje para ese campo" }
}
```

4.3 Contrato del API

Implementa exactamente estas rutas, con el privilegio exigido y los códigos de estado indicados.

Verbo	Ruta	Privilegio	Códigos	Observación
POST	/api/v1/auth/login	Público	200 · 401	Devuelve token, nombre y privilegios.
GET	/api/v1/lotes?pagina&tamanio&valle	LOTE_LEER	200 · 401	Listado paginado del padrón.
GET	/api/v1/lotes/{id}	LOTE_LEER	200 · 401 · 404	Detalle de un lote.
GET	/api/v1/lotes/{id}/labores?pagina&tamanio	LABOR_LEER	200 · 401 · 404	Labores del lote, de la más reciente a la más antigua.
POST	/api/v1/labores	LABOR_CREAR	201 · 400 · 401 · 403 · 409	Registra una labor. Aplica RN-01 y RN-04.
PUT	/api/v1/labores/{id}	LABOR_EDITAR	200 · 400 · 401 · 403 · 404 · 409	Corrige una labor. Aplica RN-01 y RN-02.
DELETE	/api/v1/labores/{id}	LABOR_ELIMINAR	204 · 401 · 403 · 404	Elimina una labor. Sin cuerpo en la respuesta.

Verbo	Ruta	Privilegio	Códigos	Observación
POST	/api/v1/lotes/{id}/cerrar-campania	CAMPANIA_CERRAR	200 · 401 · 403 · 404 · 409	Cierra la campaña. Aplica RN-03.
GET	/api/v1/usuarios/me	Autenticado	200 · 401	Datos y privilegios del usuario del token.

4.4 Aplicación AgroCampo

Ámbito	Exigencia
Cliente Ktor	Una única instancia de HttpClient inyectada con Koin, con negociación de contenido, dirección base centralizada, tiempos de espera y registro reducido en producción.
DTO y dominio	DTO propios en la capa de datos, con el nombre serializado anotado cuando difiera, tolerancia a claves desconocidas y funciones de conversión al modelo de dominio. El dominio no conoce anotaciones de serialización.
CRUD completo	Las cuatro operaciones sobre el recurso principal, con estados de carga, éxito y error en la interfaz.
Errores tipados	Una jerarquía sellada de errores del dominio a la que la capa de datos traduce cada respuesta: validación, no autorizado, sin privilegio, no encontrado, conflicto, servidor, sin conexión y tiempo agotado.
Sesión y token	Token guardado con el almacenamiento seguro de Android respaldado por el almacén de claves; cabecera de autorización añadida de forma centralizada por el cliente; cierre de sesión que borra el token; recuperación de la sesión al reabrir la aplicación.
Interfaz según privilegios	Las acciones que el usuario no puede ejecutar no se ofrecen en la interfaz, y aun así el backend las rechaza si se intentan.
Persistencia local	Esquema SQLDelight en el código común, controlador de Android en la aplicación y consultas que devuelven un flujo observable. La pantalla principal lee de la base local.
Sincronización	Operación que primero envía lo pendiente y después trae lo actualizado, con marca de pendiente por fila e identificadores locales provisionales en un rango que el servidor nunca emite.
Arquitectura	Capas domain, data y presentation separadas, ViewModel con StateFlow y UiState, casos de uso, interfaz de repositorio en el dominio e inyección con Koin.
Pruebas	Al menos cuatro pruebas en commonTest con el motor HTTP simulado y el controlador de base de datos en memoria: conversión de DTO, traducción de un error, registro sin conexión y orden de la sincronización.

4.5 Modelo de dominio entregado

Implementa estas clases tal como se presentan, en el código común de la aplicación. Puedes agregar propiedades, pero no eliminar ni cambiar el tipo de las existentes.

```
// domain/model/Lote.kt
data class Lote(
    val id: Long,
    val codigo: String,
    val nombre: String,
    val hectareas: Double,
    val cultivo: String,
    val valle: String,
    val campaniaCerrada: Boolean,
    val dosisMaximaPorHectarea: Double
)

// domain/model/TipoLabor.kt
enum class TipoLabor { RIEGO, FUMIGACION, COSECHA }

// domain/model/Labor.kt
data class Labor(
    val id: Long,              // negativo mientras esté pendiente de envío
    val loteId: Long,
    val tipo: TipoLabor,
    val fecha: String,         // ISO-8601: 2026-10-27
    val responsable: String,
    val cantidad: Double,
    val observacion: String,
    val pendiente: Boolean     // true mientras no haya llegado al servidor
)

// domain/model/Sesion.kt
data class Sesion(
    val token: String,
    val usuario: String,
    val rol: String,
    val privilegios: Set<String>
)
```

Por qué el identificador es negativo mientras está pendiente

El identificador definitivo lo asigna el servicio, y sin conexión no se puede pedir. La fila debe guardarse igual, de modo que se le asigna un valor de un rango que el servicio nunca emite. Así no colisiona con ningún identificador real y permite reconocer de un vistazo qué filas faltan enviar. Al sincronizar, la fila provisional se sustituye por la definitiva dentro de la misma transacción.

4.6 Esquema de la base de datos local

Se te entrega el esquema y las dos consultas que resuelven la parte menos evidente. Complétalo con el resto de consultas que necesites.

```
-- sqldelight/pe/upeu/agrosierra/AgroDb.sq
CREATE TABLE lote (
    id INTEGER NOT NULL PRIMARY KEY,
    codigo TEXT NOT NULL, nombre TEXT NOT NULL,
    hectareas REAL NOT NULL, cultivo TEXT NOT NULL,
    valle TEXT NOT NULL, campaniaCerrada INTEGER NOT NULL,
    dosisMaxima REAL NOT NULL
);

CREATE TABLE labor (
    id INTEGER NOT NULL PRIMARY KEY,
    loteId INTEGER NOT NULL, tipo TEXT NOT NULL,
    fecha TEXT NOT NULL, responsable TEXT NOT NULL,
    cantidad REAL NOT NULL, observacion TEXT NOT NULL,
    pendiente INTEGER NOT NULL
);

observarLaboresPorLote:
SELECT * FROM labor WHERE loteId = ? ORDER BY fecha DESC;

-- COALESCE evita el NULL de la tabla vacia; el minOf de Kotlin
-- garantiza que el id provisional sea siempre negativo.
siguienteIdLocal:
SELECT COALESCE(MIN(id), 0) FROM labor;
```

4.7 Estructura de paquetes esperada

Servicio:

```
src/main/kotlin/pe/upeu/agrosierra/
├── config/          SecurityConfig.kt, JwtAuthenticationFilter.kt,
│                    JwtService.kt, ManejadorDeErrores.kt
├── seguridad/       Usuario.kt, Rol.kt, Privilegio.kt,
│                    UsuarioRepository.kt, AuthController.kt
├── dominio/         entidades y reglas de negocio del caso
├── repositorio/     acceso a la base de datos
├── servicio/        reglas RN-01 a RN-04 y transacciones
├── web/             controladores REST bajo /api/v1
```

```
│   └── dto/          peticiones y respuestas, con el envoltorio
│                     de paginacion y el formato de error comun
└── Aplicacion.kt
```

Aplicación:

```
shared/src/commonMain/kotlin/pe/upeu/agrosierra/
├── domain/
│   ├── model/          modelo de la seccion 4.5 y ErrorApi.kt
│   ├── repository/     interfaces (solo contratos)
│   └── usecase/        casos de uso, incluido SincronizarUseCase.kt
├── data/
│   ├── remote/         ClienteHttp.kt, ApiConfig.kt, MapeoErrores.kt
│   │   └── dto/        DTO y funciones de conversion al dominio
│   ├── local/          AlmacenSeguro.kt (interfaz), BaseLocal.kt
│   └── repository/     implementaciones offline-first
├── presentation/       sesion/, listado/, detalle/, perfil/,
│                       navigation/ y theme/
└── di/                 dataModule, domainModule, presentationModule,
                        redModule y platformModule

androidApp/src/main/kotlin/pe/upeu/agrosierra/
├── AlmacenSeguroAndroid.kt   preferencias cifradas + almacen de claves
├── BaseLocalAndroid.kt       driver de SQLDelight para Android
└── PlatformModule.android.kt motor HTTP y dependencias de Android
```

5. Versionado y trazabilidad con Git

El desarrollo del producto es individual, pero igualmente debe quedar versionado y ser trazable en los dos repositorios: el del servicio y el de la aplicación. No se acepta un proyecto subido en un único commit al final: el historial debe mostrar cómo creció el trabajo y cuándo. Esta condición vale dos puntos de la nota final.

> **Regla no negociable**
> El historial debe acreditar que el producto lo construiste tú, a lo largo de la unidad. Un repositorio sin ramas de funcionalidad, con el código volcado en uno o dos commits del último día, no acredita autoría y se califica con cero en este criterio, aunque el producto funcione perfectamente.

5.1 Modelo de ramas

Ambos repositorios se organizan con el siguiente esquema mínimo, el mismo que usarías en un equipo. Ninguna funcionalidad se desarrolla directamente sobre main.

Rama	Propósito	Quién escribe en ella
main	Código estable y presentable. El backend levanta y la aplicación compila y se ejecuta en Android.	Nunca se trabaja aquí: solo recibe fusiones desde develop.
develop	Integración del trabajo antes de llevarlo a main.	Se actualiza por fusión desde las ramas de funcionalidad.
feature/<funcionalidad>	Desarrollo de una funcionalidad concreta: seguridad, CRUD, offline, pruebas.	Una rama por funcionalidad, no una rama por día.
fix/<descripción>	Corrección de un defecto detectado durante la integración.	Se abre desde develop y vuelve a develop.
sc-<letra>-<apellido>	Solicitud de cambio de la Parte II; mínimo tres commits.	El estudiante evaluado, durante la sesión.

5.2 Flujo de trabajo obligatorio

1. Planifica el trabajo por funcionalidades y regístralo como issues o tareas en cada repositorio: seguridad, CRUD, persistencia local, sincronización y pruebas.
2. Antes de empezar cada jornada, sitúate en develop y actualiza tu copia local.
3. Crea una rama de funcionalidad a partir de develop. Ninguna funcionalidad se desarrolla sobre main.
4. Trabaja en commits pequeños y frecuentes: al menos un commit por sesión de trabajo, nunca un único commit con todo.
5. Cuando la funcionalidad esté terminada y compile, abre una solicitud de incorporación hacia develop y descríbela en dos o tres líneas: qué resuelve y qué probaste.
6. Fusiona con --no-ff para que el historial conserve la rama y se vea qué entró y cuándo.
7. Resuelve los conflictos en la rama de funcionalidad, no en develop.
8. Al cierre de la unidad, fusiona develop en main y etiqueta el commit evaluado como v1.0-unidad2 en ambos repositorios.

```
# Inicio de una funcionalidad
git checkout develop
git pull origin develop
git checkout -b feature/seguridad-jwt

# Trabajo diario
git add .
git commit -m "feat(seguridad): exigir privilegio en el endpoint de registro"
git push -u origin feature/seguridad-jwt

# Cierre de la unidad
git checkout main
```

```
git merge --no-ff develop
git tag -a v1.0-unidad2 -m "Producto Unidad 2"
git push origin main --tags
```

5.3 Convención de mensajes de commit

Los mensajes se escriben en español, en presente y describiendo el efecto del cambio, no la acción mecánica.

Prefijo	Cuándo se usa	Ejemplo
feat	Se agrega una funcionalidad nueva.	feat(sincronizacion): subir pendientes antes de bajar
fix	Se corrige un defecto.	fix(token): borrar el token al recibir 401
refactor	Se reorganiza el código sin cambiar el comportamiento.	refactor(datos): traducir los errores HTTP en un solo lugar
test	Se agregan o corrigen pruebas.	test(offline): registrar sin red y verificar la marca de pendiente
docs	Documentación o README.	docs: describir el contrato del API y los roles

Mensajes como «cambios», «avance», «actualización» o «subida final» no cumplen la convención y restan puntaje en el criterio de trazabilidad.

5.4 Evidencia que se revisará

- Gráfico de ramas de cada repositorio (git log --graph --oneline --all o la vista de red de la plataforma), con varias ramas de funcionalidad y sus fusiones.
- Historial con commits distribuidos a lo largo de la unidad, verificable con git log --date=short --pretty="%h %ad %s".
- Al menos tres solicitudes de incorporación cerradas en cada repositorio, con su descripción.
- La etiqueta v1.0-unidad2 apuntando al commit evaluado en el backend y en la aplicación.
- Ausencia de commits directos sobre main que no provengan de una fusión.

Cómo se califica la trazabilidad con Git (2 puntos)

2 puntos: ramas de funcionalidad en los dos repositorios, commits distribuidos a lo largo de la unidad, fusiones por solicitud de incorporación y mensajes que cumplen la convención.

1 punto: hay ramas, pero los commits se concentran en uno o dos días, las fusiones son directas o los mensajes son genéricos.

0 puntos: se trabajó directamente sobre main, el historial tiene uno o dos commits, o los repositorios se crearon el día de la evaluación.

6. Parte II. Solicitud de cambio del cliente

Durante la evaluación, la dirección «solicita» un ajuste. El docente asigna una solicitud distinta a cada estudiante, que dispone de ochenta minutos para implementarla en vivo. Varias solicitudes atraviesan el servicio y la aplicación: se evalúa que el cambio funcione de extremo a extremo y que respete la separación de capas.

Código	Solicitud de cambio	Qué se observa
SC-A	Agregar al padrón un filtro «Solo campañas abiertas» que se combine con el filtro de valle y funcione también sin conexión.	Que el filtro se resuelva en la consulta local o en el caso de uso, no recorriendo la lista dentro del composable.
SC-B	Incorporar al servicio y a la aplicación un rol nuevo, SUPERVISOR_CALIDAD, con los privilegios de lectura más LABOR_EDITAR sobre todas las labores, pero sin CAMPANIA_CERRAR.	Que el rol se agregue donde viven los privilegios y que ni el filtro de seguridad ni los controladores necesiten un if con el nombre del rol.
SC-C	Añadir el campo «cantidad de jornales» a la labor, en el servicio, en el contrato, en la base local y en las dos pantallas donde aparece.	Que el cambio recorra el backend, el DTO, la conversión al dominio, el esquema local y la interfaz de forma coherente, y que los datos semilla se actualicen.
SC-D	Mostrar en el padrón un distintivo con el número de labores pendientes de sincronizar por lote, que se actualice solo al registrar o al sincronizar.	Que el número provenga de una consulta observable de la base local y no de un contador mantenido a mano en el ViewModel.

Cómo se califica la Parte II

Funciona de extremo a extremo y respeta la arquitectura: 4 puntos. Funciona pero con lógica de negocio o de seguridad mal ubicada o duplicada: 2,5 puntos. Cambio incompleto o que rompe otra funcionalidad: 1 punto. No implementado: 0 puntos.

La solicitud de cambio se desarrolla en una rama con el nombre sc-<letra>-<apellido>, creada a partir de develop en cada repositorio que toque, y se sube antes de finalizar la sesión. Un cambio que no quede versionado no se califica.

Mínimo de commits durante el examen

Cada estudiante debe subir al menos tres commits durante los ochenta minutos del bloque, separados en el tiempo y con mensajes que sigan la convención de la sección 5.3. Un criterio razonable de corte es un commit cuando el cambio compila, otro cuando la funcionalidad opera y un tercero cuando queda limpia y verificada.

Un estudiante que entregue su solicitud de cambio en un único commit final pierde un punto de la Parte II, aunque el cambio funcione correctamente.

7. Parte III. Banco de preguntas de defensa

El docente elige dos preguntas por estudiante. Responder correctamente exige mostrar el código propio y el sistema en ejecución, no solo describirlo.

1. Muéstrame dónde se verifica el privilegio de esta operación en el backend y explica por qué no basta con ocultar el botón en la aplicación.
2. Abre el token que devuelve tu inicio de sesión y explícame qué transporta, quién lo firma y qué ocurre exactamente cuando vence.
3. ¿Dónde se guarda el token en el dispositivo y por qué ahí? ¿Qué pasaría si lo guardaras en preferencias sin cifrar?
4. Muéstrame el punto único donde la cabecera de autorización se añade a las peticiones. ¿Qué ocurriría si lo hicieras función por función?
5. Explica la diferencia entre el 401 y el 403 en tu aplicación y muéstrame el código que los distingue.
6. Enséñame el recorrido completo de un registro desde la tabla del backend hasta el pixel que se dibuja, nombrando cada archivo por el que pasa.
7. ¿Por qué tu pantalla principal observa la base de datos local y no la respuesta del servicio? Muéstrame la consulta y el flujo.
8. Registra algo con el modo avión activado y explícame qué quedó guardado, cómo lo marcaste y qué identificador recibió.
9. Ejecuta la sincronización y explícame en qué orden ocurre. ¿Qué pasaría si invirtieras ese orden?
10. Muéstrame tus pruebas de commonTest: qué sustituyes para la base de datos, qué para la red y qué comportamiento garantizan.
11. Provoca un error de validación desde la aplicación y muéstrame el camino del mensaje hasta el campo del formulario.
12. ¿Qué tendrías que tocar para agregar un rol nuevo con un privilegio adicional? Nómbrame los archivos uno por uno.
13. Muéstrame tu rama de trabajo y explica qué aportaste tú al producto, qué conflicto tuviste que resolver y cómo lo resolviste.

8. Rúbrica de evaluación

El examen se califica sobre veinte puntos. Los cinco primeros criterios corresponden a los criterios de evaluación del Producto de la Unidad 2 declarados en el sílabo: consumo REST con Ktor, CRUD completo, manejo de estados y errores, autenticación JWT y persistencia local con sincronización offline-first.

Criterio	Pts.	Logrado (100 %)	En proceso (60 %)	Inicial (30 %)
Consumo REST con Ktor	2	Cliente único inyectado, DTO separados del dominio, listados paginados consumidos correctamente y dirección base centralizada.	Consume el servicio, pero crea el cliente por petición o usa el modelo de dominio como DTO.	El consumo falla o la aplicación depende de datos escritos a mano.
CRUD completo sobre el servicio	2	Las cuatro operaciones funcionan contra el backend propio y el resultado se refleja en la base de datos del servidor.	Dos o tres operaciones funcionan, o alguna no refleja el cambio en el servidor.	Solo funciona la consulta.
Manejo de estados y errores	2	Estados de carga, contenido, vacío y error; errores tipados que distinguen validación, 401, 403, 404, 409, servidor y falta de red, con conductas distintas en la interfaz.	Estados presentes pero el error es genérico, o no distingue el 401 del 403.	Un único mensaje de error para todo o la pantalla queda bloqueada en el indicador de progreso.
Autenticación JWT, roles y privilegios	3	Inicio de sesión que firma el token, filtro que protege todas las rutas, verificación por privilegio, token en almacenamiento seguro, cabecera centralizada y cierre de sesión que borra el token.	La autenticación funciona, pero el privilegio se verifica solo en la interfaz, el token se guarda sin cifrar o la cabecera se añade función por función.	Las rutas quedan abiertas, el token no se valida o no hay distinción de roles.
Persistencia local y sincronización offline-first	3	La interfaz lee de la base local mediante un flujo observable, la aplicación funciona con el modo avión, lo registrado sin red queda pendiente y la sincronización sube antes de bajar sin duplicar ni perder filas.	Hay base local, pero la pantalla sigue dependiendo de la red o la sincronización pierde o duplica alguna fila.	No hay persistencia local o la aplicación queda inutilizable sin conexión.
Trazabilidad del desarrollo con Git	2	Ramas de funcionalidad en ambos repositorios, commits distribuidos en el tiempo, fusiones por solicitud de incorporación y mensajes según la convención.	Hay ramas, pero los commits se concentran en pocos días, las fusiones son directas o los mensajes son genéricos.	Trabajo sobre main, historial de uno o dos commits o repositorios creados el día de la evaluación.

Criterio	Pts.	Logrado (100 %)	En proceso (60 %)	Inicial (30 %)
Solicitud de cambio (Parte II)	4	El cambio funciona de extremo a extremo, está ubicado en la capa correcta y llegó en al menos tres commits.	El cambio funciona con lógica duplicada, mal ubicada o en un único commit final.	Cambio incompleto o que rompe otra funcionalidad.
Defensa técnica (Parte III)	2	Responde con precisión y sustenta sus decisiones mostrando el código y la aplicación en ejecución.	Responde parcialmente o describe sin mostrar el código.	No logra explicar el funcionamiento de su propio producto.

Nota mínima aprobatoria

Conforme al sílabo, la nota mínima aprobatoria de la asignatura es trece. Un estudiante que no logre levantar su servicio durante la evaluación no puede superar los doce puntos, cualquiera sea su desempeño en las demás partes, porque sin servicio no hay nada que consumir ni que proteger.

9. Entregables

Cada estudiante entrega, al término de la sesión:

- Enlace al repositorio del backend con la etiqueta «v1.0-unidad2» sobre el commit evaluado.
- Enlace al repositorio de la aplicación Android con la misma etiqueta.
- Documento PDF con capturas de las pantallas en ejecución, incluidas las del inicio de sesión, la operación rechazada por falta de privilegio y la aplicación funcionando con el modo avión activado.
- Colección de peticiones del API exportada desde la herramienta que usaron para probarla, con los ejemplos de 401, 403 y 400.
- Archivo README.md en cada repositorio: estructura de paquetes, contrato del API, roles y privilegios, decisiones de arquitectura e instrucciones de ejecución.
- Rama con la solicitud de cambio de la Parte II («sc-a-apellido», «sc-b-apellido», etc.).
- Captura del gráfico de ramas de ambos repositorios, incorporada al documento PDF de evidencias.
- Resultado de la ejecución de las pruebas de commonTest.

9.1 Lista de cotejo previa a la evaluación

Verifica estos puntos antes de presentarte. Un «No» en cualquiera de ellos implica perder puntaje.

N.º	Criterio observable	Cumple (Sí / No)
1	El backend levanta desde el repositorio entregado con un solo comando y responde en la dirección indicada.	
2	El inicio de sesión devuelve un token firmado y rechaza credenciales incorrectas.	

N.º	Criterio observable	Cumple (Sí / No)
3	Todas las rutas salvo la de inicio de sesión responden 401 cuando no se envía el token.	
4	Un token alterado o vencido produce 401 y no 200.	
5	Un usuario sin el privilegio necesario recibe 403 y la sesión no se cierra.	
6	Los listados devuelven el envoltorio de paginación y aceptan página y tamaño.	
7	Un formulario inválido responde 400 con el detalle por campo.	
8	La aplicación guarda el token con el almacenamiento seguro de Android.	
9	La cabecera de autorización se añade de forma centralizada y no función por función.	
10	Al cerrar sesión el token se borra y la aplicación vuelve al inicio de sesión.	
11	Las acciones no permitidas para el rol no se ofrecen en la interfaz.	
12	La pantalla principal se alimenta de la base de datos local mediante un flujo observable.	
13	La aplicación muestra datos con el modo avión activado.	
14	Un registro creado sin red queda marcado como pendiente y con identificador local provisional.	
15	Tras sincronizar, ese registro llegó al servidor y no está duplicado ni desapareció.	
16	Las pruebas de commonTest se ejecutan y pasan.	
17	Cada repositorio tiene varias ramas de funcionalidad y commits distribuidos en varias fechas.	
18	Ninguna funcionalidad se subió directamente a main: todo llegó por fusión desde develop.	
19	Los mensajes de commit siguen la convención y el commit evaluado está etiquetado como v1.0-unidad2.	
20	Durante el examen registré al menos tres commits en mi rama, no uno solo al final.	

10. Anexo. Usuarios y datos semilla

El servicio debe cargar estos datos al iniciar, de modo que la demostración pueda ejecutarse sin preparación manual previa.

```
-- Usuarios de prueba (la contrasena va cifrada en la base;
-- aqui se escribe en claro solo para el examen)
tecnico1  / Agro2026*   ROL TECNICO
jefe1     / Agro2026*   ROL JEFE_CAMPO
admin     / Agro2026*   ROL ADMINISTRADOR
```

```
-- Datos minimos exigidos
8 lotes en 3 valles: al menos 2 con la campania ya cerrada.
20 labores repartidas: cada lote abierto con 2 o mas labores,
   al menos un lote con labor de tipo COSECHA y otro sin ella
   (para poder demostrar RN-03 en los dos sentidos).
3 usuarios, uno por rol, con sus privilegios asignados.
```

Por qué estos mínimos

El anexo exige estos mínimos para que la demostración sea posible: sin un lote con campaña cerrada no se puede evidenciar la regla RN-01, y sin un lote sin cosecha no se puede evidenciar la RN-03.